Cheaper Re-planning, not Less Re-planning: An Efficiency Study of SmolVLA on a Turing GPU

Minh-Ngoc Luong
Hanoi University of Science and Technology
ngoclm004@gmail.com

Code: https://github.com/Ngoc-LM/efficient-vla

Abstract

A vision-language-action (VLA) policy that re-plans at every control step pays for the whole model at every step. For the released LIBERO checkpoint of SmolVLA on a Tesla T4, this report measures where the cost goes and which reductions keep closed-loop success on identical, paired episodes. A re-plan takes 789 ms, dominated by emulated bfloat16 and a launch-bound 10-step flow loop. Re-planning less often (open-loop chunks, gripper-event triggers, a cached image) lowers success by about 12 points. Re-planning more cheaply keeps it: float16 with 3 Euler steps is non-inferior on 500 paired episodes (78.4% vs 75.4%, 95% CI of the difference $[-0.6, +6.6]$ points), and with a CUDA-graph replay that returns the same actions a re-plan takes 112 ms ($7.1\times$); one Euler step (93 ms, $8.5\times$) is non-inferior on 300 fresh episodes. One flow noise per episode removes gripper chatter but lowers success from 77.0% to 23.5%.

1. Introduction

VLA policies map images, a language instruction and the robot state to actions through a pretrained vision-language model (VLM) and an action head [1, 17]. At batch size one inside a control loop, every re-plan waits for the full model. Efficient-VLA work cuts this cost in the model, the decoding, the execution or the runtime (Sec. 2), but a saving can change closed-loop behaviour: an ONNX export of this SmolVLA checkpoint cut median latency by 40% and, through truncated instructions, success from 70% to 41% [6].

This report studies SmolVLA [17] on LIBERO-Spatial [12] with a Tesla T4, a GPU without bfloat16 tensor cores, without training: weights and protocol stay fixed, only the execution changes. It surveys efficient VLAs and diffusion versus flow matching (Sec. 2; 10-paper literature matrix), reproduces the released checkpoint (Sec. 3), analyses it (Sec. 4), and tests on identical episodes whether to re-plan less often or more cheaply (Sec. 5). The levers that make re-planning cheaper are known [13, 19]; the contribution is a measured account of why each helps on this hardware and a paired closed-loop test of whether success is preserved.

Lever	Reported effect, by paper (hardware)
Smaller model	SmolVLA [17]: 0.45B, LIBERO avg 87.3. TinyVLA [20]: 14 vs 292 ms per action against OpenVLA (A6000).
Fewer tokens or layers	VLA-Cache [21]: $1.63\times$ (4090). EfficientVLA [22]: $1.93\times$ at 28.9% FLOPs, -0.6 SR (A40). DeeR [23]: 5.2–$6.5\times$ fewer LLM FLOPs (trained).
Decoding	FAST [15]: up to $5\times$ faster training, 750 vs 100 ms per chunk. OFT [7]: $26\times$ throughput. CogACT [10]: small DiT head.
Execution	RTC [2]: tolerates $>$300 ms delay, +21 ms per chunk. A2C2 [16]: trained 4.7 ms head.
Systems	Realtime-VLA [13]: π_0 106.5$\rightarrow$27.3 ms against naive PyTorch (4090). ONNX [6]: -40% latency, lower SR. Coda [19]: 2 steps + learned corrector.

Table 1. Efficiency levers, as reported by each paper.

2. Efficient VLAs and Action Generation

Efficiency levers. Tab. 1 groups efficiency work by the part of the inference loop it changes. Token and layer reduction assumes redundancy across frames [21] or easy situations [23] and saves compute only in the language model. Chunked and asynchronous execution assumes that older actions stay valid; SmolVLA's asynchronous mode was 30% faster but lowered success from 78.3% to 73.3% [17]. Systems work assumes the model stays numerically the same, which an export can violate [6]. Two gaps remain: speed-ups come with aggregate success rates, though a non-significant difference is not equivalence, and most latencies with GPUs that have native bfloat16.

Diffusion versus flow matching. Both turn Gaussian noise $\boldsymbol{\epsilon}$ into an action chunk $\mathbf{A}$ conditioned on the observation $\mathbf{o}$. Diffusion heads [3, 10] noise the data as $\mathbf{A}_k = \sqrt{\bar{\alpha}_k}\mathbf{A} + \sqrt{1-\bar{\alpha}_k}\boldsymbol{\epsilon}$ and predict the noise [4]; flow-matching heads [1, 17] interpolate linearly, $\mathbf{A}^\tau = \tau\boldsymbol{\epsilon} + (1-\tau)\mathbf{A}$, and regress

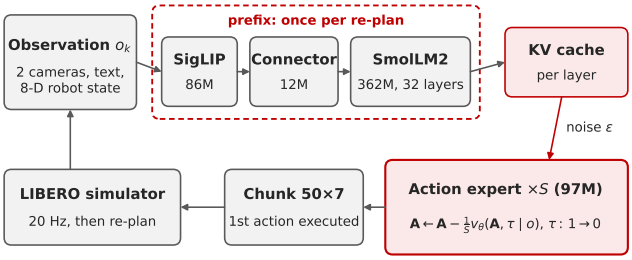


Figure 1. One re-plan. Two 512^2 images, the instruction and the state form a prefix processed once; the expert, trained with $\mathcal{L}_{\text{FM}}$ (Eq. (2), $\tau \sim \text{Beta}(1.5, 1)$ on $[0.001, 1]$), attends to its cache in each of S=10 Euler steps; the first of 50 actions (delta pose, gripper) is executed.

the velocity of that path [11]:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{A}_k, k \mid \mathbf{o})\right\|^2, \tag{1}$$

$$\mathcal{L}_{\text{FM}} = \mathbb{E}\left\|(\boldsymbol{\epsilon} - \mathbf{A}) - v_\theta(\mathbf{A}^\tau, \tau \mid \mathbf{o})\right\|^2. \tag{2}$$

They differ in four respects. (i) Target: noise versus velocity. (ii) Path: the linear path gives each training pair a constant velocity and yields straighter sampling trajectories [11]; on RoboTwin, $\pi_{0.5}$ keeps 71.3% success with 3 flow steps against 71.5% with 10, and drops to 59.8% with one [19]. (iii) Sampler: Diffusion Policy uses 100 DDPM steps in simulation and 10 DDIM [18] steps on the robot, CogACT 10 DDIM steps with guidance, flow models 10 Euler steps. (iv) Architecture: Diffusion Policy repeats its whole denoiser; CogACT, π_0 and SmolVLA run the VLM once and repeat only a head on its cached output.

3. Reproduction

Model and setup. The study runs the released LeRobot implementation of SmolVLA (v0.6.1) with the checkpoint `HuggingFaceVLA/smolvla_libero` on Kaggle (2× Tesla T4, PyTorch 2.11, MuJoCo 3.3.7, EGL rendering). The checkpoint differs from the paper configuration [17]: it keeps all 32 decoder layers of SmolVLM2-500M instead of 16, uses an action expert of width 0.5× instead of 0.75×, and executes one action per predicted chunk (n_{exec}=1). It has 605M parameters, stored in bfloat16 (Fig. 1).

Metric and protocol. Success rate (SR) is the fraction of episodes whose goal predicate (*e.g.* `On(bowl, plate)`) holds within 280 steps. An episode is a (task, stored initial state) pair; the 10 tasks run on initial states 0–19 (200 episodes; 0–49 for the main comparison). Methods are compared on identical episodes with common random numbers: the flow noise at step k is seeded by (episode, k).

Differences and result. MuJoCo is pinned below 3.4, from which the stored initial states of task 5 settle differently and

Component (one re-plan)	GFLOP	bf16 (ms)	fp16 (ms)
Vision encoder, 2 cameras	431	230	39
Prefix incl. vision (32 LM layers)	527	304	82
One expert pass (flow step)	11.5	51	52
Re-plan, S=10 (released)	642	814	605
Simulator step (CPU + EGL)	–	32	

Table 2. Cost of one re-plan on a Tesla T4 (isolated components, CUDA events, median of 60 calls; Tab. 3, from a separate run, agrees within 4% for a full re-plan).

SmolVLA drops from 80% to 28% on that task [9]. The reproduced SR of 77.0% [70.7, 82.3] (Wilson 95% CI) is below the 90% reported for the paper configuration but inside the 57–83% obtained by others with this checkpoint [6, 8].

4. Analysis

Computational bottlenecks. Tab. 2 breaks down one re-plan; the simulator is not the bottleneck (32 ms per step against 814 ms for the policy). The vision encoder takes 230 ms in the released bfloat16 weights but 39 ms in float16: Turing GPUs have no bfloat16 tensor cores and emulate it. The action expert accounts for 18% of the FLOPs of a re-plan but 63% of its latency (10 × 51 ms), and its step time is the same in both precisions: a pass over 50 action tokens is only 11.5 GFLOP, and replaying its kernels from a CUDA graph cuts it from 53 to 9.4 ms (Sec. 5): at batch one the GPU waits on kernel launches rather than on arithmetic.

Successful and failed trajectories. Successes finish in a median of 100 steps; all 46 failures run to the 280-step limit after at least one gripper closure. From the executed gripper commands (object poses are not logged; segments of at most 3 steps merged), 9 failures close the gripper three or more times, 9 others keep it closed at the limit, and 28 reopen it after one or two closures; task 3 (bowl on the cookie box) fails in 10 of 20 episodes. Fresh flow noise at every re-plan makes consecutive chunks differ about as much as independent samples (first normalised action: 1.31 from the shifted previous chunk, 1.37 between two noises; 5 LIBERO-Object episodes). The gripper command chatters accordingly: failed episodes contain 12.6 gripper segments of at most 3 steps per 100 steps, successful ones 2.7.

Strengths. **S1 Small footprint:** 1.16 GB of weights and 21 MB of activations per re-plan (float16) fit a 16 GB T4 several times. **S2 Prefix caching:** the expert reuses the cached prefix, so a flow step costs 52 ms against 304 ms for the prefix. **S3 Few-step flow:** with S=3 the first action deviates from S=10 by 0.32, a quarter of the sampling spread (1.37), and SR holds down to S=1 (Sec. 5).

	bfloat16 (released)		float16	
Euler steps S	eager	graph	eager	graph
10 (released)	789	572	616	178
3	433	372	245	**112**
2	386	345	191	103
1	335	318	136	93
Fit $a+bS$	$284+50.5S$	$289+28.3S$	$84+53.3S$	$84+9.4S$

Table 3. Latency of one re-plan (ms, median of 60 calls, T4, batch 1); least-squares fit over S (residuals $\leq$2.7 ms).

Limitations. **L1 Precision mismatch:** bfloat16 weights run emulated on Turing. **L2 Launch-bound expert:** the flow loop dominates latency at batch one while using little arithmetic. **L3 Need for fresh images:** executing 5 actions per chunk, or re-planning with fresh proprioception but an image up to 4 steps old, loses 12.0–12.5 points (Sec. 5), so an episode needs a full re-plan at every step. **L4 A fresh sample at every step:** independent noise at every re-plan produces gripper chatter, concentrated in failures, although removing it alone does not raise SR measurably (Sec. 5).

5. Improvement: Re-plan Cheaper, not Less

Bottleneck and hypotheses. **H1 (re-plan less):** computation can be shared across steps without lowering SR, by executing k actions per chunk (open loop), by also re-planning when the next queued action switches the gripper, or by re-planning with fresh proprioception on image and language keys and values cached for up to 5 steps and refreshed after gripper switches (stale image). **H2 (re-plan more cheaply):** each re-plan can be made cheaper while computing the same function up to precision and discretisation: float16 removes the emulated arithmetic (same weights, projections kept in float32), S=3 removes discretisation slack (S3 in Sec. 4), and a CUDA graph removes the launches of the expert loop, which should gain more once precision is fixed. For the graph, the S-step Euler loop is captured once per S with static buffers and replayed (the prefix stays eager); on one observation its actions equal the eager ones exactly in all 8 settings. Two probes test whether the chatter of L4 matters, by reusing one flow noise for every re-plan of an episode (fixed noise), and how much the executed action depends on the noise for each S.

Protocol. Differences in SR are paired (exact McNemar test, Newcombe 95% CI [14], Holm adjustment [5] over the nine comparisons of Tab. 4; the two tests on states 20–49 below are unadjusted); a variant is non-inferior if the lower bound exceeds -5 points, a margin fixed before any run. SR comes from eager runs; latency is measured on isolated calls, because these runs share each GPU between two processes, and checked in the loop with one process per GPU.

Variant	Vis./100	SR	ΔSR	95% CI	p_{Holm}
Released (bf16, S=10)	100	77.0	–	–	–
H1: re-plan less					
Open loop, k=5	20.1	64.5	−12.5	[−20.3, −4.5]	0.018
Open loop, k=10	10.2	67.5	−9.5	[−17.3, −1.6]	0.124
k=5 + gripper events	21.5	65.0	−12.0	[−18.7, −5.2]	0.007
Stale image (k=5, S=3)	24.6	65.0	−12.0	[−19.1, −4.7]	0.013
H2: re-plan more cheaply					
bf16, S=3	100	79.0	+2.0	[−3.8, +7.8]	1.000
fp16, S=3	100	77.0	0.0	[−5.8, +5.8]	1.000
fp16, S=2	100	77.5	+0.5	[−5.7, +6.7]	1.000
fp16, S=1	100	79.5	+2.5	[−4.2, +9.2]	1.000
Probe: fixed noise, S=3	100	23.5	−53.5	[−60.3, −45.4]	<0.001

Table 4. Closed-loop results on 200 paired episodes (initial states 0–19); Holm over the nine comparisons. Vis./100: vision-encoder passes per 100 control steps.

Re-planning less lowers success (H1 rejected). Calling the vision encoder about five times less often costs 12.0–12.5 points of SR (Tab. 4); with k=10 the drop of 9.5 points is not significant after adjustment. The event trigger removes most chatter (1.2 short gripper segments per 100 steps, against 5.0) but not the loss; fresh proprioception does not replace a current image.

Re-planning more cheaply preserves success (H2). Along released → float16 → S=3 → graph, a re-plan takes 789, 616, 245 and 112 ms, factors of 1.28, 2.51 and 2.19 (7.1× in total; Tab. 3). By the fits, an Euler step costs 51–53 ms eagerly in either precision (launch-bound) but 28.3 and 9.4 ms with the graph, which exposes the emulated arithmetic: the graph gains 1.38× in bfloat16 and 3.46× in float16 at S=10. On 200 pairs the lower bound for float16 S=3 (-5.8) misses the margin, so states 20–49 were added: on these 300 pairs alone it succeeds in 79.3% against 74.3% (+5.0 points, CI $[+0.3, +9.7]$), on all 500 in 78.4% against 75.4% (+3.0, CI $[-0.6, +6.6]$; task bootstrap $[-1.2, +8.4]$): non-inferior in both. Fewer steps do not lower SR measurably either (S=2: 77.5%, S=1: 79.5%); on the same 300 pairs, in a test fixed before the run, S=1 succeeds in 80.3% against 74.3% (+6.0, CI $[+0.6, +11.4]$; task bootstrap $[-1.0, +12.7]$) at 93 ms (8.5×): non-inferior, with a gain close to that of S=3 and not robust across tasks. On RoboTwin, where SmolVLA executes 50 actions per chunk, one step lowers success from 64.4% to 35.0% [19]; re-planning at every step may correct what one step gets wrong. In the loop, a float16 S=3 re-plan takes 275 ms eagerly and 114 ms with the graph (released: 862 ms); replay keeps the first action and the outcome of all 20 episodes, and whole trajectories match in 10, as often as two eager runs: execution is not bit-reproducible.

Consistent noise is not the remedy. Fixed noise removes the chatter (0.5 against 3.8 short gripper segments per 100

steps) but lowers SR from 77.0% to 23.5% (111 against 4 discordant pairs, $p < 10^{-27}$), whereas $S{=}1$ removes it (0.7) without lowering SR, so chatter reads as a symptom rather than the main cause. Fresh noise gives errors that average out over steps and fixed noise repeats one error, whereas $S{=}1$ returns $\boldsymbol{\epsilon} - v_\theta(\boldsymbol{\epsilon}, 1 \mid \mathbf{o})$, which at the minimiser of Eq. (2) is $\mathbb{E}[\mathbf{A} \mid \mathbf{o}]$ for any $\boldsymbol{\epsilon}$ (a mean over modes if $p(\mathbf{A} \mid \mathbf{o})$ is multimodal): on 193 observations, the executed action varies across 8 noises by 0.18 with $S{=}1$, against 0.71 with $S{=}3$ and 1.00 with $S{=}10$.

Trade-offs and next bottleneck. The float16 gain needs a GPU without bfloat16 tensor cores (elsewhere only the step and graph gains apply, not measured); float16 changes actions by 0.010 on average (one observation), and the graph needs static shapes and one capture per S; with 32 ms of simulation, the loop runs at about 7 Hz (control rate 20 Hz). The prefix (84 ms, still eager) takes 75–90% of a re-plan; further gains must come from it, through training-free kernel fusion or token reuse [13, 21] (not tested here) or a smaller retrained backbone.

6. Conclusion

On a T4, the cost of a SmolVLA re-plan is set less by FLOPs than by emulated bfloat16 and a launch-bound flow loop. Sharing computation across steps lowers success by about 12 points; removing these overheads and the discretisation slack with known levers makes a re-plan $7.1\times$ cheaper ($8.5\times$ with one Euler step) and stays non-inferior on paired episodes. One noise per episode removes chatter and most successes, so chatter reads as a symptom. **Limitations:** one checkpoint, suite and GPU type, in simulation; the 500-pair test was extended after an interim look; baseline 77.0% against 90% for the paper configuration. **Future work:** a cheaper prefix (kernel fusion, token reuse or a distilled backbone) and the same paired closed-loop test on embedded GPUs and exported runtimes [6].

References

[1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1

[2] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *Advances in Neural Information Processing Systems*, 38:33383–33407, 2026. 1

[3] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025. 1

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 1

[5] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian journal of statistics*, pages 65–70, 1979. 3

[6] Rafiqul Islam. When faster VLA deployment changes closed-loop behavior: Task success-latency analysis of SmolVLA across PyTorch and ONNX variants. *arXiv preprint arXiv:2609.14146*, 2026. 1, 2, 4

[7] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 1

[8] LeRobot contributors. Cannot reproduce SmolVLA results on LIBERO benchmark. GitHub issue huggingface/lerobot #2354, 2025. Accessed October 2026. 2

[9] LeRobot contributors. mujoco>=3.4.0 silently breaks the initial-state premise of libero_spatial task 5. GitHub issue huggingface/lerobot #4390, 2026. Accessed October 2026. 2

[10] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024. 1

[11] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. 2

[12] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023. 1

[13] Yunchao Ma, Yizhuang Zhou, Yunhuan Yang, Tiancai Wang, and Haoqiang Fan. Running VLAs at real-time speed. *arXiv preprint arXiv:2510.26742*, 2025. 1, 4

[14] Robert G Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in medicine*, 17(22):2635–2650, 1998. 3

[15] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025. 1

[16] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. Leave no observation behind: Real-time correction for VLA action chunks. *arXiv preprint arXiv:2509.23224*, 2025. 1

[17] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 1, 2

[18] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020. 2

[19] Zhipeng Tang, Xinda Chen, Weining Rao, Xiao Li, Wenting Tan, Yuning Wang, Xiao Shi, and Xiaofang Zhao. Fewer steps, better actions: Rethinking flow-matching inference for VLA policies. *arXiv preprint arXiv:2609.21216*, 2026. 1, 2, 3

[20] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. TinyVLA: toward fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 10 (4):3988–3995, 2025. 1

[21] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching. *Advances in Neural Information Processing Systems*, 38:164448–164473, 2026. 1, 4

[22] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. EfficientVLA: Training-free acceleration and compression for vision-language-action models. *Advances in Neural Information Processing Systems*, 38: 40891–40914, 2026. 1

[23] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. DeeR-VLA: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37: 56619–56643, 2024. 1